Automation & CI/CD Integration

Pipeline-Driven Migration at Scale

Manual VM migration doesn't scale beyond 20 VMs. hyper2kvm's YAML-driven approach integrates with Ansible, Terraform, Jenkins, GitLab CI, and GitHub Actions for fully automated, repeatable, auditable migration pipelines. Migrate 500+ VMs with approval gates, rollback, and observability.

YAML Config | Ansible | Terraform | GitLab CI | Jenkins | Batch Orchestration | Prometheus

YAML-Driven Migration

Declarative configuration for reproducible, version-controlled migrations.

Single VM Configuration

```
# migration.yaml — single VM conversion
source:
  vmdk: /data/exports/db-server.vmdk
  # Or: ova: /data/exports/db-server.ova
  # Or: vhd: /data/exports/db-server.vhdx

output:
  directory: /var/lib/libvirt/images/
  format: qcow2
  compress: true

fixes:
  fstab: true
  initramfs: true
  network: true

deploy:
  emit_domain_xml: true
  deploy_libvirt: true
  boot_test: true
  memory: 8192
  vcpus: 4
  network_bridge: br0
```

Batch Migration Configuration

```
# batch-migration.yaml — multiple VMs in dependency order
global:
  output_directory: /var/lib/libvirt/images/
  format: qcow2
  compress: true
  fixes: { fstab: true, initramfs: true }
  deploy: { emit_domain_xml: true, deploy_libvirt: true }

vms:
  - name: postgres-primary
    source: { vmdk: /data/exports/postgres-primary.vmdk }
    deploy: { memory: 16384, vcpus: 8, network_bridge: br-db }
    priority: 1    # Migrated first (database)
```

```
- name: app-server-1
  source: { vmdk: /data/exports/app-server-1.vmdk }
  deploy: { memory: 8192, vcpus: 4, network_bridge: br-app }
  depends_on: [postgres-primary]
  priority: 2

- name: app-server-2
  source: { vmdk: /data/exports/app-server-2.vmdk }
  deploy: { memory: 8192, vcpus: 4, network_bridge: br-app }
  depends_on: [postgres-primary]
  priority: 2

- name: nginx-lb
  source: { vmdk: /data/exports/nginx-lb.vmdk }
  deploy: { memory: 4096, vcpus: 2, network_bridge: br0 }
  depends_on: [app-server-1, app-server-2]
  priority: 3    # Migrated last (load balancer)
```

Dry Run & Validation

`h2kvmctl convert --config batch-migration.yaml --dry-run` — validates YAML syntax, checks source files exist, verifies disk space, validates network bridges, reports dependency order, and estimates total conversion time. Always dry-run before production migration.

Ansible Playbook Integration

Inventory-driven migration across multiple hosts with idempotent operations.

Ansible Inventory

Inventory (hosts.yml)

```yaml
all:
  children:
    kvm_hosts:
      hosts:
        kvm-host-01:
          ansible_host: 10.0.1.11
          migration_vms:
            - name: db-server
              source: /nfs/exports/db-server.vmdk
              memory: 16384
              vcpus: 8
              bridge: br-db
            - name: app-server
              source: /nfs/exports/app-server.vmdk
              memory: 8192
              vcpus: 4
              bridge: br-app
        kvm-host-02:
          ansible_host: 10.0.1.12
          migration_vms:
            - name: web-server
              source: /nfs/exports/web.vmdk
              memory: 4096
              vcpus: 2
              bridge: br0
```

Playbook (migrate.yml)

```yaml
---
- name: Migrate VMs to KVM
  hosts: kvm_hosts
  become: true
  tasks:
    - name: Install hyper2kvm
      pip:
        name: hyper2kvm
        state: present

    - name: Run doctor check
      command: h2kvmctl doctor
      register: doctor_result
      changed_when: false

    - name: Convert each VM
      command: >
        h2kvmctl convert
          --source {{ item.source }}
          --output /var/lib/libvirt/images/
          --format qcow2 --compress
          --fix-fstab --fix-initramfs
          --emit-domain-xml
          --deploy-libvirt
          --memory {{ item.memory }}
          --vcpus {{ item.vcpus }}
          --network-bridge {{ item.bridge }}
      loop: "{{ migration_vms }}"
      register: conversion_results

    - name: Verify VMs are running
      command: virsh domstate {{ item.name }}
      loop: "{{ migration_vms }}"
      register: vm_states
      failed_when: >
        vm_states.stdout != "running"
```

Rolling Migration with Ansible

```
# Run migration in waves with serial execution
- name: Wave 1 - Database tier
  hosts: kvm_hosts
  serial: 1                    # One host at a time
  max_fail_percentage: 0       # Stop on any failure
  tasks:
    - include_tasks: migrate-vm.yml
      loop: "{{ migration_vms | selectattr('tier', 'equalto', 'database') }}"

- name: Wave 2 - Application tier
  hosts: kvm_hosts
  serial: 2                    # Two hosts in parallel
  tasks:
    - include_tasks: migrate-vm.yml
      loop: "{{ migration_vms | selectattr('tier', 'equalto', 'application') }}"

# Execute: ansible-playbook -i hosts.yml migrate.yml --check  (dry run)
#          ansible-playbook -i hosts.yml migrate.yml           (real run)
```

Terraform Provider Concepts

Infrastructure-as-code for migration state tracking and post-migration provisioning.

Migration + Libvirt Terraform Workflow

Pre-Migration: Convert with h2kvmctl

```
# Terraform null_resource for conversion
resource "null_resource" "migrate_vm" {
  for_each = var.vms_to_migrate

  provisioner "local-exec" {
    command = <<-EOT
      h2kvmctl convert \
        --source ${each.value.source} \
        --output /var/lib/libvirt/images/ \
        --format qcow2 --compress \
        --fix-fstab --fix-initramfs \
        --json > /tmp/${each.key}.json
    EOT
  }

  triggers = {
    source_checksum = filesha256(
      each.value.source
    )
  }
}
```

Post-Migration: Libvirt Provider

```
# Deploy converted VMs with libvirt provider
terraform {
  required_providers {
    libvirt = {
      source = "dmacvicar/libvirt"
    }
  }
}

resource "libvirt_volume" "vm_disk" {
  for_each = var.vms_to_migrate
  name   = "${each.key}.qcow2"
  pool   = "default"
  source = "/var/lib/libvirt/images/${each.key}.qcow2"
  format = "qcow2"
  depends_on = [null_resource.migrate_vm]
}

resource "libvirt_domain" "vm" {
  for_each = var.vms_to_migrate
  name   = each.key
  memory = each.value.memory
  vcpu   = each.value.vcpus
  disk { volume_id = libvirt_volume.vm_disk[each.key].id }
  network_interface { bridge = each.value.bridge }
}
```

Terraform State Benefits

Feature	Manual Migration	Ansible	Terraform
State tracking	Spreadsheet	Partial (facts)	Full state file
Drift detection	Manual audit	Re-run detects	terraform plan

Rollback	**Manual**	Playbook needed	**terraform destroy**
Dependency graph	**Manual ordering**	Serial/handlers	**Automatic DAG**
Idempotent	**No**	**Yes**	**Yes**
Multi-provider	**No**	Modules	**Native (DNS, LB, K8s)**

CI/CD Pipeline Templates

Ready-to-use pipeline configs for GitLab CI, Jenkins, and GitHub Actions.

GitLab CI Pipeline

```
# .gitlab-ci.yml — VM Migration Pipeline
stages: [validate, export, convert, test, deploy]

variables:
  VCENTER_HOST: vcenter.corp.com
  OUTPUT_DIR: /var/lib/libvirt/images

validate:
  stage: validate
  script:
    - h2kvmctl doctor
    - h2kvmctl convert --config migration.yaml --dry-run
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

export:
  stage: export
  script:
    - h2kvmctl export --server $VCENTER_HOST --vm "$VM_LIST" --output /data/exports/
  artifacts:
    paths: [/data/exports/*.vmdk]
    expire_in: 7 days
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual  # Approval gate

convert:
  stage: convert
  script:
    - h2kvmctl convert --config migration.yaml --json > results.json
  artifacts:
    paths: [results.json]
    reports:
      dotenv: results.env

test:
  stage: test
  script:
    - h2kvmctl boot-test --config migration.yaml --timeout 120
    - ./scripts/smoke-test.sh  # Application-level tests
  allow_failure: false
```

```
deploy:
  stage: deploy
  script:
    - h2kvmctl deploy --config migration.yaml --deploy-libvirt
    - virsh list --all
  environment:
    name: production
  when: manual  # Final approval gate
```

GitHub Actions Workflow

```
# .github/workflows/migrate.yml
name: VM Migration
on:
  workflow_dispatch:
    inputs:
      vm_list: { description: 'VMs to migrate', required: true }
      dry_run: { description: 'Dry run only', type: boolean, default: true }
jobs:
  migrate:
    runs-on: self-hosted  # Must run on KVM host
    steps:
      - uses: actions/checkout@v4
      - name: Validate
        run: h2kvmctl convert --config migration.yaml --dry-run
      - name: Convert
        if: ${{ !inputs.dry_run }}
        run: h2kvmctl convert --config migration.yaml --json | tee results.json
      - name: Upload results
        uses: actions/upload-artifact@v4
        with: { name: migration-results, path: results.json }
```

Batch Migration Orchestration

Migrating 100-500+ VMs in waves with dependency ordering and parallel execution.

Migration Wave Planning

Wave	VMs	Tier	Parallel	Duration	Dependencies	Rollback Window
Wave 0	5	Dev/Test	5 parallel	2 hours	None	Immediate (no impact)
Wave 1	10	Infrastructure (DNS, DHCP, AD)	2 serial	4 hours	None	4 hours
Wave 2	20	Database servers	4 parallel	6 hours	Wave 1	2 hours
Wave 3	50	Application servers	10 parallel	8 hours	Wave 2	2 hours
Wave 4	30	Web / Load Balancer	10 parallel	4 hours	Wave 3	1 hour
Wave 5	15	Monitoring / Backup	5 parallel	3 hours	Wave 4	4 hours

Total: 130 VMs in 5 waves over 2 weekends

Parallel Execution with GNU Parallel

```
# Convert 10 VMs in parallel (limited by I/O bandwidth)
cat vm-list.txt | parallel -j 10 \
  h2kvmctl convert --source /data/exports/{}.vmdk \
    --output /var/lib/libvirt/images/ \
    --format qcow2 --compress \
    --fix-fstab --fix-initramfs \
    --emit-domain-xml \
    --json '>' /tmp/results/{}.json

# Monitor progress
watch -n 5 'ls /tmp/results/*.json | wc -l; echo "of $(wc -l < vm-list.txt) total"'

# Aggregate results
jq -s '[.[] | {name: .vm_name, status: .status, duration: .duration_seconds}]' \
  /tmp/results/*.json > migration-report.json
```

Rollback Procedure

- Keep VMware VMs powered off (not deleted) for 48 hours

Progress Dashboard

- Real-time: `h2kvmctl status --config batch.yaml`

- Rollback = power on VMware VM, power off KVM VM
- DNS/MAC: preserved, so no network changes needed
- Data: use VM snapshot before cutover for point-in-time
- Automation: `h2kvmctl rollback --wave 3`
- Decision deadline: 2 hours after go-live per wave

- JSON output for custom dashboards
- Fields: VM name, status, progress %, ETA, errors
- Webhook: POST to Slack/Teams on completion or failure
- Email: summary report per wave
- Grafana: Prometheus metrics for visual tracking

API & Scripting Interface

Python API and JSON output for custom automation scripts.

Python Scripting

Subprocess Integration

```
import subprocess
import json

def migrate_vm(source, output_dir,
               memory=4096, vcpus=2):
    """Migrate a single VM with h2kvmctl."""
    cmd = [
        "h2kvmctl", "convert",
        "--source", source,
        "--output", output_dir,
        "--format", "qcow2",
        "--compress",
        "--fix-fstab",
        "--fix-initramfs",
        "--emit-domain-xml",
        "--deploy-libvirt",
        "--memory", str(memory),
        "--vcpus", str(vcpus),
        "--json"
    ]
    result = subprocess.run(
        cmd, capture_output=True, text=True
    )
    if result.returncode != 0:
        raise RuntimeError(result.stderr)
    return json.loads(result.stdout)

# Usage
result = migrate_vm(
    "/data/exports/web-server.vmdk",
    "/var/lib/libvirt/images/",
    memory=8192, vcpus=4
)
print(f"Migrated {result['vm_name']} "
      f"in {result['duration_seconds']}s")
```

Batch Script with Error Handling

```
import concurrent.futures
import json
from pathlib import Path

VMS = [
    {"name": "db-server",
     "source": "/data/db-server.vmdk",
     "memory": 16384, "vcpus": 8},
    {"name": "app-server",
     "source": "/data/app-server.vmdk",
     "memory": 8192, "vcpus": 4},
    {"name": "web-server",
     "source": "/data/web-server.vmdk",
     "memory": 4096, "vcpus": 2},
]

results = {"success": [], "failed": []}

with concurrent.futures.ThreadPoolExecutor(
    max_workers=4
) as executor:
    futures = {
        executor.submit(
            migrate_vm,
            vm["source"],
            "/var/lib/libvirt/images/",
            vm["memory"], vm["vcpus"]
        ): vm["name"]
        for vm in VMS
    }
    for future in concurrent.futures.as_completed(
        futures
    ):
        name = futures[future]
        try:
            r = future.result()
            results["success"].append(name)
            print(f"OK: {name}")
```

```
        except Exception as e:
            results["failed"].append(
                {"name": name, "error": str(e)}
            )
            print(f"FAIL: {name}: {e}")

# Summary
print(json.dumps(results, indent=2))
```

Exit Codes & JSON Output

Exit Code	Meaning	JSON Field	Action
0	Success	`"status": "success"`	Proceed to next VM
1	General error	`"status": "error", "error": "..."`	Check error message, retry or skip
2	Source not found	`"error": "source_not_found"`	Verify file path, check NFS mount
3	Insufficient disk space	`"error": "disk_space"`	Free space or change output dir
4	Conversion failed	`"error": "conversion_failed"`	Check source image integrity
5	Deploy failed	`"error": "deploy_failed"`	Check libvirt, verify XML

Monitoring & Observability

Prometheus metrics, Grafana dashboards, and structured logging for migration runs.

Prometheus Metrics

```
# h2kvmctl exposes metrics on port 9090 during batch runs
# h2kvmctl convert --config batch.yaml --metrics-port 9090

# Metrics endpoint: http://localhost:9090/metrics
# Prometheus scrape config:
# scrape_configs:
#   - job_name: 'h2kvmctl'
#     static_configs:
#       - targets: ['kvm-host-01:9090']

# Available metrics:
h2kvmctl_migrations_total{status="success"}        42
h2kvmctl_migrations_total{status="failed"}          2
h2kvmctl_migration_duration_seconds_bucket{le="60"} 10
h2kvmctl_migration_duration_seconds_bucket{le="300"} 35
h2kvmctl_migration_duration_seconds_bucket{le="900"} 42
h2kvmctl_disk_bytes_converted_total                  2.1e+12
h2kvmctl_disk_conversion_rate_bytes_per_second       150e+6
h2kvmctl_active_conversions                          4
h2kvmctl_queue_depth                                 8
```

Structured Logging

JSON Log Format

```
{
  "timestamp": "2026-03-23T10:15:30Z",
  "level": "info",
  "event": "conversion_complete",
  "vm_name": "db-server",
  "source_format": "vmdk",
  "source_size_gb": 120,
  "output_size_gb": 45,
  "compression_ratio": 0.375,
  "duration_seconds": 340,
  "fixes_applied": [
    "fstab", "initramfs", "network"
  ],
```

Alerting Rules

```
# Prometheus alerting rules
groups:
  - name: h2kvmctl
    rules:
      - alert: MigrationFailed
        expr: >
          increase(
            h2kvmctl_migrations_total
            {status="failed"}[5m]
          ) > 0
        labels:
          severity: critical
        annotations:
```

```
  "deploy_status": "running",
  "host": "kvm-host-01"
}
```

```
    summary: "VM migration failed"

- alert: SlowConversion
  expr: >
    h2kvmctl_migration_duration_seconds
    > 1800
  labels:
    severity: warning
  annotations:
    summary: "Migration taking >30min"

- alert: DiskSpaceLow
  expr: >
    node_filesystem_avail_bytes
    {mountpoint="/var/lib/libvirt"}
    < 50e9
  labels:
    severity: critical
```

Enterprise Workflow Examples

Three real-world automation scenarios with cost and time comparisons.

Scenario Comparison

Scenario	VMs	Manual Time	Automated Time	Manual Cost	Automated Cost	Savings
Nightly Sync	20	N/A (impossible)	2 hrs/night	N/A	$0/night	Enables DR
DC Migration	200	6 months / 3 FTEs	3 weekends	$450,000	$45,000	$405,000 (90%)
Multi-Site	500	12 months / 5 FTEs	6 weekends	$1,200,000	$90,000	$1,110,000 (92%)

Scenario 1: Nightly VMware→KVM Sync

- **Goal:** DR copy of 20 critical VMs on KVM
- Cron job exports changed VMDKs nightly via govc
- h2kvmctl converts incremental changes
- KVM VMs updated, ready for instant failover
- RPO: 24 hours (nightly), RTO: 5 minutes (boot KVM)
- Pipeline: GitLab CI scheduled pipeline, Slack alerts
- Ansible verifies boot + connectivity after each sync
- **Zero manual effort after initial setup**

Scenario 2: 200-VM Data Center Migration

- **Goal:** Complete VMware exit in 3 weekends
- Week 1: Export all 200 VMs (automated, govc batch)
- Weekend 1: Wave 0-1, dev/test + infrastructure (35 VMs)
- Weekday: Validate, fix issues, prep Wave 2-3
- Weekend 2: Wave 2-3, databases + apps (100 VMs)
- Weekend 3: Wave 4-5, web + monitoring (65 VMs)
- Jenkins pipeline with manual approval gates per wave
- **3 weekends vs 6 months manual**

Scenario 3: Multi-DC Rolling Migration

- **Goal:** 500 VMs across 5 data centers
- Terraform manages cross-DC state and dependencies
- Each DC migrated independently in 1 weekend
- Ansible handles per-host configuration
- GitHub Actions with environment-specific approvals
- Grafana dashboard: global migration progress
- Prometheus alerts: failure rate, conversion speed
- **6 weekends vs 12 months manual**

Automation Stack Recommendations

- **<20 VMs:** Shell script + h2kvmctl YAML
- **20-100 VMs:** Ansible playbook + h2kvmctl
- **100-300 VMs:** GitLab CI + Ansible + Prometheus
- **300+ VMs:** Terraform + Ansible + CI/CD + Grafana
- **Air-gapped:** Ansible (runs offline) + local Gitea
- **Always:** --dry-run first, approval gates, rollback plan

Automation Reduces Migration Cost by 90% at Scale

Manual migration at 2 hours/VM means 200 VMs = 400 hours = 50 person-days. Automated pipeline at 6 minutes/VM means 200 VMs = 20 hours unattended. The pipeline pays for itself after 10 VMs. Start with YAML + dry-run, add CI/CD for anything over 20 VMs.